Experiment Number: 01

Experiment Name: University System.

Objectives:

- To understand how Inheritance work.
- To understand how access specifiers works to access data across classes .

Theory:

In OOPS Inheritance allows one class to access the properties and functions of a parent class. It helps to reuse code without rewriting it. Access specifiers like private, protected, and public control how data is accessed across the classes. Private members are only accessible inside that specific class, protected members are accessible within the class and its derived classes, and public members are accessible from anywhere.

Algorithm:

1. Start program.
2. Create base class Person.
3. Create derived class Student.
4. Create another derived class GraduateStudent.
5. Add functions to set and show all data.
6. Take input, call functions, and show result.

Code:

t here | *University_System.cpp

```cpp
#include <iostream>
using namespace std;
class Person {
private:
    string name;
    int age;
public:
    void setPerson(string n, int a) {
        name = n;
        age = a;}
    void showPerson( ) {
        cout << "Name: " << name << endl;
        cout << "Age: " << age << endl;}
};
class Student : public Person {
protected:
    int rollNumber;
public:
    void setStudent(int r) {
        rollNumber = r;}
    void showStudent( ) {
        cout << "Roll Number: " << rollNumber << endl;}
};
class GraduateStudent : public Student {
private:
    string researchTopic;
public:
    void setGraduateStudent(string topic) {
        researchTopic = topic;}
    void showGraduateStudent() {
        showPerson( );
        showStudent( );
        cout << "Research Topic: " << researchTopic << endl;}
};
int main() {
    GraduateStudent gs;
    gs.setPerson("Rezwan", 20);
    gs.setStudent(102);
    gs.setGraduateStudent("Software Engineering");
    cout << "Graduate Student Information: " << endl << endl;
    gs.showGraduateStudent( );
    return 0;
}
```

Output:

```
"D:\Versity Work\BAUST CSE\
Graduate Student Information:

Name: Rezwan
Age: 20
Roll Number: 102
Research Topic: Software Engineering
```

Conclusion:

In this experiment we have successfully saw how the base class Person securely stored personal data as private, the derived class Student used protected roll number to allow controlled access, and the final GraduateStudent added its own private data.

Experiment Number: 02

Experiment Name: Savings account in a bank .

Objectives:

- To create a Banking system using oops to show deposit withdraw and total savings.
- To understand how Multiple Inheritance work in C++.

Theory:

In OOPS when a class can inherit characteristics, data members and member functions from multiple base classes is called Multiple Inheritance. It is beneficial for code reusability, combining different functionalities, modeling real-world roles and for to understand the code easily.

Algorithm:

1. Start the program.
2. Create a base class Account with balance, and functions for deposit and withdraw.
3. Create SavingsAccount derived from Account with interestRate and calculateInterest.
4. Create PremiumSavings derived from SavingsAccount with bonusAmount and displayTotalSavings.
5. Create an object of PremiumSavings, perform deposit, withdraw, and show total savings.
6. End program.

Code:

```
#include <iostream>
using namespace std;
class Account {
protected:
    double balance;
public:
    Account(double b = 0) {
        balance = b;}
    void deposit(double amount) {
        balance += amount;
        cout << "Deposited: " << amount << endl;}
    void withdraw(double amount) {
        if (amount <= balance) {
            balance -= amount;
            cout << "Withdrawn: " << amount << endl;}
            else {cout << "Insufficient balance!" << endl;}}
    double getBalance( ) {
        return balance;}
};
class SavingsAccount : public Account {
protected:
    double interestRate;
public:
    SavingsAccount(double b = 0, double rate = 0.05) : Account(b) {
        interestRate = rate;}
    double calculateInterest( ) {
        return balance * interestRate;}
};
class PremiumSavings : public SavingsAccount {
private:
    double bonusAmount;
public:
    PremiumSavings(double b = 0, double rate = 0.05, double bonus = 100) {
        balance = b;
        interestRate = rate;
        bonusAmount = bonus;}
    void displayTotalSavings( ) {
        double interest = calculateInterest( );
        double total = getBalance( ) + interest + bonusAmount;
        cout << "\nBalance: " << getBalance( ) << endl;
        cout << "Interest: " << interest << endl;
        cout << "Bonus: " << bonusAmount << endl;
        cout << "Total Savings: " << total << endl;}
};
int main( ) {
    PremiumSavings ps(8000, 0.1, 500);
    ps.deposit(1000);
    ps.withdraw(500);
    ps.displayTotalSavings( );
    return 0;
```

Output:

Conclusion:

In this program we have successfully solved the Bank Account problem and showed the deposited and withdrawn and total saving using OOPS. By using 3 classes Account, SavingAccount, PremiumSavings and used the object of PremiumSavings to execute the code and show output.

Experiment Number: 03

Experiment Name: Use of Multiple Inheritance and Constructor-destructor.

Objectives:

- To understand how to use Inheritance and Multiple Inheritance in OOPS .
- To understand how the execution flow of Constructor and Destructor works.

Theory:

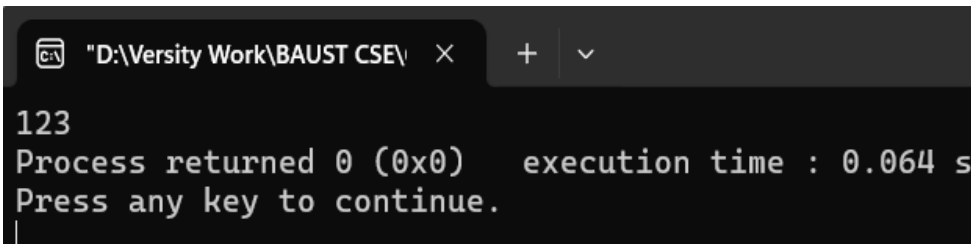
The difference between Inheritance and Multiple Inheritance is basic inheritance child class inherit members and functions from only from one parent class but in multiple inheritance a child class inherits members and functions from multiple parent classes or more than one class. Constructors are executed in the order of inheritance 1st base class, then derived class, while destructors are executed in reverse order 1st derived class first, then base class.

Code 1:

here | Sunday_Class.cpp | Sunday_Class2.cpp

```cpp
#include <iostream>
using namespace std;
class base {
protected :
   int a, b;
public :
   void setab (int n, int m){
   a = n;
   b = m;
   }
};
class derived : public base {
   int c;
public :
   void setc(int n){c = n;}
   void showabc ( ){
   cout << a << b << c;
   }
};
int main ( ){
   derived obj;
   obj.setab (1,2);
   obj.setc (3);
   obj.showabc ( );
   return 0;
}
```

Output:



Code 2:

here | Sunday_Class.cpp | *Sunday_Class2.cpp

```cpp
#include <iostream>
using namespace std;
class base {
protected :
   int a, b;
public :
   void setab (int n, int m){
   a = n;
   b = m;
   }
};
class derived : protected base {
   int c;
public :
   void setc(int n){c = n;}
   void showabc ( ){
   cout << a << b << c;
   }
};
int main ( ){
   derived obj;
   //obj.setab (1,2);  //It cant be accessed because it is inherited in protected
   obj.setc (3);
   obj.showabc ( );
   return 0;
}
```

Output:

```
"D:\Versity Work\BAUST CSE\
003
Process returned 0 (0x0)   execution time : 0.061
Press any key to continue.
```

Code3 :

t here | Sunday_Class.cpp | Sunday_Class2.cpp | Sunday_Class3.cpp

```cpp
#include<iostream>
using namespace std;
class base{
public:
   base( ){
   cout<< " Constructing base "<<endl;
   }
   ~base( ){
      cout<< " Destructing base "<<endl;
   }
};
class derived:public base{
public:
   derived( ){
      cout<< " Constructing derived "<<endl;
   }
   ~derived( ){
      cout<<" Destructing derived "<<endl;
   }
};

int main( ){
derived obj;
return 0;
}
```

Output:

```
"D:\Versity Work\BAUST CSE\
Constructing base
Constructing derived
Destructing derived
Destructing base
```

Code 4:

here | Sunday_Class.cpp | Sunday_Class2.cpp | Sunday_Class3.cpp | *Tuesday_Class.cpp

```cpp
#include <iostream>
using namespace std;
class base {
    int i;
public:
    base(int n) {
        cout << "Constructing base" << endl;
        i = n;
    }
    ~base( ) {
        cout << "Destructing base" << endl;
    }
    void showi( ) {
        cout << i << endl;
    }
};
class derived : public base {
    int j;
public:
    derived(int n) : base(n) {
        cout << "Constructing derived" << endl;
        j = n;
    }
    ~derived( ) {
        cout << "Destructing derived" << endl;
    }
    void showj( ) {
        cout << j << endl;
    }
};
int main( ) {
    derived obj(10);
    obj.showi( );
    obj.showj( );
    return 0;
}
```

Output:

```
Constructing base
Constructing derived
10
10
Destructing derived
Destructing base
```

Code 5:

here | Sunday_Class.cpp | Sunday_Class2.cpp | Sunday_Class3.cpp | *Tuesday_Class.cpp | Tuesday_HW.cpp

```cpp
#include <iostream>
using namespace std;
class A {
    int x;
public:
    A (int a) {
        cout << "Constructing A" << endl;
        x = a; }
    ~A ( ) {
        cout << "Destructing A" << endl; }
    void showx( ) {
        cout << x << endl;}
};
class B : public A {
    int y;
public:
    B (int a, int b) : A (a) {
        cout << "Constructing B" << endl;
        y = a; }
    ~B ( ) {
        cout << "Destructing B" << endl; }
    void showy( ) {
        cout << y << endl; }
};
class C : public B {
    int z;
public:
    C (int a, int b, int c) : B (a, b) {
        cout << "Constructing C" << endl;
        z = b; }
    ~C ( ) {
        cout << "Destructing C" << endl; }
    void showz( ) {
        cout << z << endl; }
};
int main( ) {
    C obj(1, 2, 3);
    obj.showx( );
    obj.showy( );
    obj.showz( );
    return 0;
}
```

Output:

```
Constructing A
Constructing B
Constructing C
1
1
2
Destructing C
Destructing B
Destructing A
```

Conclusion:

In this experiment we have learned how to use Multiple Inheritance and how the execution flow of Constructor and Destructor works. This helps us to understand the concepts off OOPS better in C++.